

FC7xxx PWM Integration Manual

Rev.0.6

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.3	2023/09/27	Updated for MCAL V0.3.0
0.4	2023/12/15	Updated for MCAL V0.4.0 - Added new section in memmap and exclusive area in code
0.6	2023/03/29	Updated for MCAL V0.6.0 - Added support for FC7240

Table of Contents

Revision History	2
Table of Contents	3
Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory	7
3.1 Sections in Memory Map	7
Chapter 4 Exclusive Area	9
Chapter 5 Interrupt Service Routine (ISR)	11
Chapter 6 Error Report	12
6.1 Det	12
6.2 Dem	13
Chapter 7 Function Calls	14
7.1 Function Calls during Startup	14
7.2 Function Calls during Shutdown	14
7.3 Function Calls during Wake-up	14
7.4 Function Calls during Runtime	14
Chapter 8 Other Requirements	15
8.1 Notification, Callback, Callout	15
8.2 Macros	15
Chapter 9 Integration Steps	16

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the PWM module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for the PWM module.
- Det: This module is necessary for enabling Development error detection.
- EcuC: This module provides configuration for mapping PWM drivers to ECUC partitions (when multi-core functionality is enabled)
- Port: This module provides the port configurations if want output PWM signal through pin.
- Common: This module is the basic module which used to choose the chip.

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for enabling Development error detection.

Module initialization dependency

- Mcu: To use the PMW module, the user needs to initialize the MCU module first
- Port: To output PWM signal, the user needs to initialize PORT module first

2.2 Files Required for Compile

PWM module files:

- _MCAL/Src/Pwm/Src/Pwm.c
- _MCAL/Src/Pwm/Src/Pwm_Ftu.c
- _MCAL/Src/Pwm/Src/Pwm_Hw.c
- _MCAL/Src/Pwm/include/Pwm.h
- _MCAL/Src/Pwm/include/Pwm_Ftu.h
- _MCAL/Src/Pwm/include/Pwm_Ftu_Types.h
- _MCAL/Src/Pwm/include/Pwm_Hw.h
- _MCAL/Src/Pwm/include/Pwm_Hw_Types.h
- _MCAL/Src/Pwm/include/Pwm_MemMap.h
- _MCAL/Src/Pwm/include/Pwm_version.h
- _MCAL_generate/src/Pwm_PBcfg.c
- _MCAL_generate/include/Pwm_PBcfg.h
- _MCAL_generate/include/Pwm_Cfg.h

Common module files:

- _MCAL/Src/Common/include/Aontimer_Reg.h
- _MCAL/Src/Common/include/arm_cortex_asm.h
- _MCAL/Src/Common/include/Common_MemMap.h

- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h
- _MCAL/Src/Common/include/Cpm_Reg.h
- _MCAL/Src/Common/include/Eth_GeneralTypes.h
- _MCAL/Src/Common/include/Fcpit_Reg.h
- _MCAL/Src/Common/include/Ftu_Common.h
- _MCAL/Src/Common/include/Ftu_Reg.h
- _MCAL/Src/Common/include/Ftu_RegOps.h
- _MCAL/Src/Common/include/Gpio_Reg.h
- _MCAL/Src/Common/include/IRQRouter.h
- _MCAL/Src/Common/include/Mb_Reg.h
- _MCAL/Src/Common/include/Mb_RegOps.h
- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Port_Reg.h
- _MCAL/Src/Common/include/Scm_Reg.h
- _MCAL/Src/Common/include/Scm_RegOps.h
- _MCAL/Src/Common/include/SpinLock.h
- _MCAL/Src/Common/include/StdRegMacros.h
- _MCAL/Src/Common/include/Tstmp_Reg.h
- _MCAL/Src/Common/src/Aontimer_Common.c
- _MCAL/Src/Common/src/Ftu_Common.c
- _MCAL/Src/Common/src/IRQRouter.c
- _MCAL/Src/Common/src/Port_Common.c
- _MCAL/Src/Common/src/SpinLock.c

Det module files:

- Det.h

Rte module files:

- SchM_Pwm.h

2.3 Add Plug-ins

PWM module plug-ins are developed for EB tresos Studio, so, to use PWM plug-ins on the EB tresos Studio, the user needs to add the PWM module to the EB plug-ins folder first.

- 1) Copy the PWM module(_MCAL/EB_Plugins/eclipse/plugins/Pwm) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
- 2) Set the PWM module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure the PWM module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
PWM_START_SEC_CONFIG_DATA_8 PWM_STOP_SEC_CONFIG_DATA_8 PWM_START_SEC_CONFIG_DATA_16 PWM_STOP_SEC_CONFIG_DATA_16 PWM_START_SEC_CONFIG_DATA_32 PWM_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code(data).
PWM_START_SEC_CONFIG_DATA_UNSPECIFIED PWM_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
PWM_START_SEC_CONST_BOOLEAN PWM_STOP_SEC_CONST_BOOLEAN PWM_START_SEC_CONST_8 PWM_STOP_SEC_CONST_8 PWM_START_SEC_CONST_16 PWM_STOP_SEC_CONST_16 PWM_START_SEC_CONST_32 PWM_STOP_SEC_CONST_32 PWM_START_SEC_CONST_UNSPECIFIED PWM_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
PWM_START_SEC_CODE PWM_STOP_SEC_CODE PWM_START_SEC_RAMCODE PWM_STOP_SEC_RAMCODE PWM_START_SEC_CODE_AC PWM_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code(text).
PWM_START_SEC_VAR_NO_INIT_BOOLEAN PWM_STOP_SEC_VAR_NO_INIT_BOOLEAN PWM_START_SEC_VAR_NO_INIT_8 PWM_STOP_SEC_VAR_NO_INIT_8 PWM_START_SEC_VAR_NO_INIT_16 PWM_STOP_SEC_VAR_NO_INIT_16 PWM_START_SEC_VAR_NO_INIT_32 PWM_STOP_SEC_VAR_NO_INIT_32 PWM_START_SEC_VAR_NO_INIT_UNSPECIFIED PWM_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
PWM_START_SEC_VAR_INIT_BOOLEAN PWM_STOP_SEC_VAR_INIT_BOOLEAN PWM_START_SEC_VAR_INIT_8 PWM_STOP_SEC_VAR_INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These

Section Name	Section Type	Description
PWM_START_SEC_VAR_INIT_16 PWM_STOP_SEC_VAR_INIT_16 PWM_START_SEC_VAR_INIT_32 PWM_STOP_SEC_VAR_INIT_32 PWM_START_SEC_VAR_INIT_UNSPECIFIED PWM_STOP_SEC_VAR_INIT_UNSPECIFIED		variables are initialized by startup code (data).
PWM_START_SEC_VAR_NO_INIT_BOOLEAN_NO_CACHEABLE PWM_STOP_SEC_VAR_NO_INIT_BOOLEAN_NO_CACHEABLE PWM_START_SEC_VAR_NO_INIT_8_NO_CACHEABLE PWM_STOP_SEC_VAR_NO_INIT_8_NO_CACHEABLE PWM_START_SEC_VAR_NO_INIT_16_NO_CACHEABLE PWM_STOP_SEC_VAR_NO_INIT_16_NO_CACHEABLE PWM_START_SEC_VAR_NO_INIT_32_NO_CACHEABLE PWM_STOP_SEC_VAR_NO_INIT_32_NO_CACHEABLE PWM_START_SEC_VAR_NO_INIT_UNSPECIFIED_NO_CACHEABLE PWM_STOP_SEC_VAR_NO_INIT_UNSPECIFIED_NO_CACHEABLE	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit and be placed in nocacheable section. These variables are never initialized by startup code (mcval_nocacheable_bss).
PWM_START_SEC_VAR_INIT_BOOLEAN_NO_CACHEABLE PWM_STOP_SEC_VAR_INIT_BOOLEAN_NO_CACHEABLE PWM_START_SEC_VAR_INIT_8_NO_CACHEABLE PWM_STOP_SEC_VAR_INIT_8_NO_CACHEABLE PWM_START_SEC_VAR_INIT_16_NO_CACHEABLE PWM_STOP_SEC_VAR_INIT_16_NO_CACHEABLE PWM_START_SEC_VAR_INIT_32_NO_CACHEABLE PWM_STOP_SEC_VAR_INIT_32_NO_CACHEABLE PWM_START_SEC_VAR_INIT_UNSPECIFIED_NO_CACHEABLE PWM_STOP_SEC_VAR_INIT_UNSPECIFIED_NO_CACHEABLE	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit and be placed in nocacheable section. These variables are initialized by startup code (mcval_nocacheable_data).

Chapter 4 Exclusive Area

PWM module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the PWM driver:

- Pwm_Hw.c:
 - Pwm_Hw_SetDutyCycle : exclusive area 10
 - Pwm_Hw_SetPeriodAndDuty : exclusive area 11
 - Pwm_Hw_SetOutputToldle : exclusive area 12
- Pwm_Ftu.c:
 - Pwm_Ftu_SetDutyCycle : exclusive area 0
 - Pwm_Ftu_SetPeriodAndDuty : exclusive area 1
 - Pwm_Ftu_ClearOutputForce : exclusive area 2
 - Pwm_Ftu_SetOutputToldle : exclusive area 3
 - Pwm_Ftu_DisableNotification : exclusive area 4
 - Pwm_Ftu_EnableNotification : exclusive area 5
 - Pwm_Ftu_EnableTriggerOut : exclusive area 6
 - Pwm_Ftu_DisableTriggerOut : exclusive area 7
 - Pwm_Ftu_MaskOutputs : exclusive area 8
 - Pwm_Ftu_UnMaskOutputs : exclusive area 9
- Pwm.c:
 - Pwm_Init : exclusive area 13
 - Pwm_DeInit : exclusive area 14
 - Pwm_SetDutyCycle : exclusive area 15
 - Pwm_SetPeriodAndDuty : exclusive area 16
 - Pwm_SetOutputToldle : exclusive area 17
 - Pwm_DisableNotification : exclusive area 18
 - Pwm_EnableNotification : exclusive area 19
 - Pwm_GetChannelState : exclusive area 20
 - Pwm_EnableTriggerOut : exclusive area 21
 - Pwm_DisableTriggerOut : exclusive area 22
 - Pwm_MaskOutputs : exclusive area 23
 - Pwm_UnMaskOutputs : exclusive area 24

- Pwm_SetPowerState : exclusive area 25
- Pwm_GetCurrentPowerState : exclusive area 26
- Pwm_GetTargetPowerState : exclusive area 27
- Pwm_PreparePowerState : exclusive area 28

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
FTU0	FTU0_IRQHandler	116
FTU1	FTU1_IRQHandler	117
FTU2	FTU2_IRQHandler	118
FTU3	FTU3_IRQHandler	119
FTU4	FTU4_IRQHandler	120
FTU5	FTU5_IRQHandler	121
FTU6	FTU6_IRQHandler	122
FTU7	FTU7_IRQHandler	123
FTU8	FTU8_IRQHandler	124
FTU9	FTU9_IRQHandler	125
FTU10	FTU10_IRQHandler	126
FTU11	FTU11_IRQHandler	127

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Pwm_Init	PWM_E_ALREADY_INITIALIZED; PWM_E_INIT_FAILED PWM_E_PARAM_CONFIG
Pwm_DeInit	PWM_E_UNINIT
Pwm_SetDutyCycle	PWM_E_UNINIT; PWM_E_PARAM_CHANNEL; PWM_E_DUTYCYCLE_RANGE
Pwm_SetPeriodAndDuty	PWM_E_UNINIT; PWM_E_PARAM_CHANNEL; PWM_E_DUTYCYCLE_RANGE; PWM_E_PERIOD_UNCHANGEABLE; PWM_E_PERIODVALUE
Pwm_SetOutputTogle	PWM_E_UNINIT; PWM_E_PARAM_CHANNEL;
Pwm_GetOutputState	PWM_E_UNINIT; PWM_E_PARAM_CHANNEL;
Pwm_DisableNotification	PWM_E_UNINIT; PWM_E_PARAM_CHANNEL;
Pwm_EnableNotification	PWM_E_UNINIT; PWM_E_PARAM_CHANNEL; PWM_E_PARAM_NOTIFICATION_NULL; PWM_E_PARAM_NOTIFICATION
Pwm_GetVersionInfo	PWM_E_PARAM_POINTER
Pwm_StartGlobalTime	PWM_E_UNINIT PWM_E_PARAM_GROUP
Pwm_StopGlobalTime	PWM_E_UNINIT PWM_E_PARAM_GROUP
Pwm_GetChannelState	PWM_E_UNINIT; PWM_E_PARAM_CHANNEL;
Pwm_EnableTriggerOut	PWM_E_UNINIT; PWM_E_PARAM_CHANNEL;
Pwm_DisableTriggerOut	PWM_E_UNINIT; PWM_E_PARAM_CHANNEL;
Pwm_MaskOutputs	PWM_E_UNINIT; PWM_E_PARAM_CHANNEL;
Pwm_UnMaskOutputs	PWM_E_UNINIT; PWM_E_PARAM_CHANNEL;

Function Name	Error Type
Pwm_SetPowerState	PWM_E_PARAM_POINTER; PWM_E_UNINIT; PWM_E_NOT_DISENGAGED
Pwm_GetCurrentPowerState	PWM_E_PARAM_POINTER; PWM_E_UNINIT;
Pwm_GetTargetPowerState	PWM_E_PARAM_POINTER; PWM_E_UNINIT;
Pwm_PreparePowerState	PWM_E_PARAM_POINTER; PWM_E_UNINIT;

6.2 Dem

None

Chapter 7 Function Calls

7.1 Function Calls during Startup

The API needs to be called is PWM_Init(NULL_PTR);

7.2 Function Calls during Shutdown

None

7.3 Function Calls during Wake-up

None

7.4 Function Calls during Runtime

None

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification

If the user configures notifications and the value is not "NULL_PTR" or "NULL", an extern declaration will generate in Pwm_Cfg.h. User need implement the notification in any file which include Pwm_Cfg.h.

- Callback

None

- Callout

None

8.2 Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:

AUTOSAR_OS_NOT_USED needs to be defined.

In this case, If USE_SW_VECTOR_MODE is not defined, the user needs to map the interrupt vector table function to ISR function, for example:

```
extern ISR(FTU_0_ISR);

void FTU0_IRQHandler(void)

{

    FTU_0_ISR();

}
```

- If OS is used:

Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration Steps

- 1) Configure the PWM module and generate configuration files (please refer to Building chapter for details).
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the PWM module with dependent modules.